

Algoritmos Avançados

Aulas 01 & 02: Apresentação da Disciplina & Revisão de Python

Prof. Me. Douglas Samuel Gonçalves

Fatec Bebedouro – Jorge Caram Sabbag
Tecnologia em Big Data no Agronegócio

2º Semestre / 2026

Sumário

- 1 Apresentação Geral da Disciplina
- 2 Clean Code, PEP 8 & Boas Práticas
- 3 Revisão de Python I – Memória, Tipos e Operadores
- 4 Revisão de Python II – Estruturas de Controle
- 5 Resolução Guiada de Exercícios

Quem sou eu?

- **Nome:** Douglas Samuel Gonçalves
- **Formação Acadêmica:**
 - Mestre em Ciências (Bioengenharia) – USP
 - Tecnólogo em Análise e Desenvolvimento de Sistemas
- **Atuação Profissional:**
 - Desenvolvedor / Engenheiro de Software Senior
 - Pesquisador em Visão Computacional / Inteligência Artificial

Contato & Atendimento

Dúvidas acadêmicas, suporte aos exercícios e orientações via e-mail oficial:

`douglas.goncalves01@cps.sp.gov.br`

Visão Geral da Disciplina

- **Curso:** Tecnologia em Big Data no Agronegócio
- **Disciplina:** Algoritmos Avançados (Sigla: IAL213)
- **Carga Horária Semestral:** 120 horas (6 horas/semana)
- **Linguagem Base:** Python

Objetivo Principal

Familiarizar o aluno com conceitos de programação tidos como avançados e com alto teor de abstração. Introduzir a teoria e representação computacional de grafos, otimização e complexidade por meio de aplicações desenvolvidas em Python.

Ementa e Conteúdo Programático

A disciplina está estruturada em grandes blocos temáticos:

Módulos 1 e 2: Estruturas & Busca

- Revisão Python, Convenções & Clean Code
- Tipos Abstratos de Dados (TADs): Listas, Pilhas, Dicionários, Tuplas
- Manipulação de Arquivos (Texto e Binário)
- Ordenação (Bubble, Quick) e Hashing
- Algoritmos de Busca (Sequencial e Binária)

Módulos 3 a 5: Grafos, Paradigmas & NP

- Representação e Buscas em Grafos (Largura e Profundidade)
- Caminho Mínimo, Árvore Geradora Mínima e Fluxo Máximo
- Paradigmas: Backtracking, Algoritmos Gulosos e Programação Dinâmica
- Classes de Complexidade e Teoria da NP-Completeness

Metodologia & Critérios de Avaliação

Metodologia

- Aulas expositivas e dialogadas.
- Prática em laboratório com implementação em Python.
- Resolução de problemas reais e algoritmos clássicos.

Cálculo da Média Semestral

A nota final da disciplina será dada por:

$$\text{Média Final} = \frac{P_1 + P_2}{2}$$

- P_1 : Avaliação Bimestral 1
- P_2 : Avaliação Bimestral 2

Atenção às Datas das Avaliações

Consulte o cronograma da disciplina para o planejamento das provas P_1 , P_2 , Prova Substitutiva e Prova Final.

Bibliografia Recomendada

Bibliografia Básica

- HETLAND, Magnus Lie. **Practical Python: From Novice to Professional**. Apress, 2002.
- BOAVENTURA NETTO, Paulo Oswaldo. **Grafos: Teoria, Modelos, Algoritmos**. 2. ed. LTC, 1994.
- WEST, Douglas B. **Introduction to Graph Theory**. 2. ed. Prentice Hall, 2001.

Bibliografia Complementar

- MENEZES, Nilo Ney Coutinho. **Introdução à Programação com Python**. Novatec, 2010.
- LUTZ, Mark; ASCHER, David. **Aprendendo Python**. 2. ed. Bookman, 2007.
- SZWARCFITER, Jayme Luiz. **Grafos e Algoritmos Computacionais**. Editora Campus, 1984.

Ecosistema de Desenvolvimento Python

Ferramentas Recomendadas

- **Interpretador:** Python 3.10+ instalado.
- **IDE / Editor:** VS Code ou PyCharm.
- **Extensões VS Code:** Python (Microsoft), Pylance, Jupyter.

Ambientes Virtuais (venv)

- Isolamento de dependências de projetos.
- `python -m venv .venv`
- Ativação no Linux/macOS: `source .venv/bin/activate`
- Ativação no Windows: `.venv/Scripts/activate`

A Importância do Código Legível

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

— Martin Fowler

Por que focar em Clean Code?

- Código é lido muito mais vezes do que é escrito (Guido van Rossum).
- Facilita a manutenção e depuração em projetos complexos.
- Essencial no trabalho em equipe acadêmico e profissional.

O Custo do Código Confuso

- Dificuldade de encontrar bugs em algoritmos complexos.
- Perda de eficiência em revisões de código.
- Desperdício de tempo em refatorações futuras.

O Zen do Python (`import this`)

Executar `import this` no Python revela 19 aforismos sobre filosofia de software.

Princípios Fundamentais para Algoritmos Avançados

- **Explicit is better than implicit:** Seja claro sobre o que seu código faz.
- **Simple is better than complex:** Prefira a solução simples antes de tentar abstrações desnecessárias.
- **Readability counts:** Formatação e nomes claros facilitam a revisão e depuração.
- **Errors should never pass silently:** Trate exceções explicitamente em vez de ignorá-las.

PEP 8: O Guia de Estilo Oficial

A **PEP 8** (Python Enhancement Proposal #8) define as convenções oficiais da linguagem.

Diretrizes Principais

- **Indentação:** Utilize **4 espaços** por nível de bloco (nunca misturar TABs e espaços).
- **Tamanho de Linha:** Limite máximo de 79 caracteres por linha.
- **Linhas em Branco:** 2 linhas em branco para separar funções e classes; 1 linha para métodos.
- **Espaçamento:** Espaço ao redor de operadores de atribuição ('=') e comparação ('==', '!', '!=').

PEP 8: Convenções de Nomes

Convenções de Nomes (Naming Conventions)

- **Variáveis e Funções:** snake_case
 - Ex: calcular_media, total_itens, obter_vizinhos
- **Constantes:** SNAKE_CASE_MAIUSCULO
 - Ex: TAXA_MAXIMA, PI, INFINITO
- **Classes:** PascalCase
 - Ex: GrafoValorado, NoPilha, ArvoreBinaria
- **Módulos e Arquivos:** nomes curtos em minúsculo (grafos.py, ordenacao.py).

Type Hints (Anotações de Tipo)

Introduzidas no Python 3.5+, as anotações de tipo auxiliam na auto-documentação e análise estática.

```
1 # Sem Type Hints
2 def buscar_elemento(lista, elemento):
3     return elemento in lista
4
5 # Com Type Hints (Claro, auto-documentado e seguro)
6 def buscar_elemento_otimizado(lista: list[int], elemento: int) -> bool:
7     return elemento in lista
8
9 # Estruturas mais complexas
10 def calcular_pesos(vertices: str, adjacencias: dict[str, float]) -> float:
11     return adjacencias.get(vertices, 0.0)
```

Docstrings e Documentação de Funções

Documentar o comportamento de funções é indispensável para algoritmos complexos.

```
1 def calcular_fatorial(n: int) -> int:
2     '''Calcula o fatorial de um numero inteiro nao-negativo.
3
4     Args:
5         n (int): 0 numero inteiro para o qual o fatorial sera calculado.
6
7     Returns:
8         int: 0 resultado do fatorial n!
9
10    Raises:
11        ValueError: Se n for um numero negativo.
12    '''
13    if n < 0:
14        raise ValueError("n deve ser maior ou igual a zero.")
15    return 1 if n <= 1 else n * calcular_fatorial(n - 1)
```

Exemplo de Refatoração: Antes vs. Depois

Inadequado (Sem Padrão)

```
1 def calc(x,y):  
2     a=x*0.1  
3     b=y+a  
4     return b  
  
5  
6 X=100  
7 Y=500  
8 res=calc(X,Y)  
9 print(res)
```

Clean Code (PEP 8 + Hints)

```
1 TAXA_DESCONTO = 0.10  
2  
3 def calcular_valor_total(  
4     preco_base: float, frete: float  
5 ) -> float:  
6     desconto = preco_base *  
7         TAXA_DESCONTO  
8     return frete + desconto  
9  
10 preco = 100.0  
11 valor_frete = 500.0  
12 total = calcular_valor_total(preco,  
    valor_frete)  
13 print(f"Total: R$ {total:.2f}")
```

Características Fundamentais do Python

- **Linguagem de Alto Nível e Interpretada:** O interpretador executa o código linha a linha.
- **Tipagem Dinâmica e Forte:**
 - *Dinâmica:* Não exige declaração manual do tipo da variável.
 - *Forte:* Não realiza conversões implícitas incompatíveis (ex: "10"+ 5 lança TypeError).
- **Multiparadigma:** Suporta programação procedural, orientada a objetos e funcional.

Tipos Primitivos Básicos

- **int:** Inteiros (10, -5, 0)
- **float:** Ponto flutuante (3.14159, -0.01)
- **bool:** Booleanos (True, False)
- **str:** Strings ("Algoritmos", 'Fatec')

Modelo de Memória: Variáveis são Referências

Em Python, variáveis ****não contêm valores diretamente****; elas são ***etiquetas/referências*** que apontam para objetos alocados na memória.

```
1 # a e b apontam para o mesmo objeto inteiro 100 na memória
2 a = 100
3 b = a
4
5 print(id(a))    # Exibe o endereço de memória do objeto apontado por a
6 print(id(b))    # Exibe o mesmo endereço de memória!
7
8 a = a + 1       # Cria um novo objeto 101 e reatribui 'a'
9 print(a, b)     # a = 101, b = 100
10 print(id(a) == id(b)) # False (agora apontam para objetos distintos)
```

Mutabilidade vs. Imutabilidade

Entender se um tipo é mutável ou imutável é crucial para evitar efeitos colaterais indesejados em algoritmos.

Tipos Imutáveis

Não podem ter seu estado alterado após a criação.

- `int`, `float`, `bool`
- `str`
- `tuple`

Tipos Mutáveis

Podem ter seu conteúdo modificado no mesmo endereço.

- `list`
- `dict`
- `set`

Conversão Explícita de Tipos (Casting)

```
1 # Leitura de entrada (input sempre retorna str)
2 entrada_usuario = "42"
3
4 # Conversao explicita para int e float
5 numero_int = int(entrada_usuario)
6 numero_float = float(entrada_usuario)
7
8 print(type(numero_int), numero_int * 2)      # <class 'int'> 84
9 print(type(numero_float), numero_float / 2)  # <class 'float'> 21.0
10
11 # Conversao para bool (Valores "falsy": 0, 0.0, "", [], {}, None)
12 print(bool(0))                               # False
13 print(bool("Texto"))                         # True
14 print(bool([]))                             # False
```

Operadores de Identidade: `is` vs. `==`

A Diferença Fundamental

- `==`: Compara se os **valores** de dois objetos são iguais.
- `is`: Compara se os dois objetos ocupam o **mesmo endereço de memória** (`id(a) == id(b)`).

```
1 lista1 = [1, 2, 3]
2 lista2 = [1, 2, 3]
3
4 print(lista1 == lista2)    # True (possuem os mesmos valores)
5 print(lista1 is lista2)    # False (sao duas listas distintas na memoria)
6
7 # Uso recomendado do 'is': comparacao com None
8 var = None
9 if var is None:
10     print("Variavel nao inicializada!")
```

Operadores Matemáticos e Divisão Inteira

Operadores Aritméticos

- $+$: Adição
- $-$: Subtração
- $*$: Multiplicação
- $/$: Divisão Real (retorna float)
- $//$: Divisão Inteira (descarta a parte decimal)
- $\%$: Resto da Divisão (Módulo)
- $**$: Potenciação

Exemplos Práticos

- $7 / 2 \rightarrow 3.5$
- $7 // 2 \rightarrow 3$
- $7 \% 2 \rightarrow 1$
- $2 ** 3 \rightarrow 8$

Operadores Relacionais e Lógicos

Relacionais

- ==: Igual a
- !=: Diferente de
- >: Maior que
- <: Menor que
- >=: Maior ou igual a
- <=: Menor ou igual a

Lógicos

- and: Verdadeiro se **ambos** forem verdadeiros.
- or: Verdadeiro se **ao menos um** for verdadeiro.
- not: Inverte o valor booleano.

Avaliação por Curto-Circuito (Short-Circuit Evaluation)

O Python avalia expressões lógicas da esquerda para a direita e para assim que o resultado é determinado.

```
1 # Exemplo 1: Evitando IndexError com and
2 lista = []
3 # Se len(lista) > 0 for False, lista[0] nem e avaliado!
4 if len(lista) > 0 and lista[0] == 10:
5     print("Primeiro elemento e 10")
6
7 # Exemplo 2: Evitando ZeroDivisionError
8 divisor = 0
9 if divisor != 0 and 100 / divisor > 5:
10     print("Resultado valido")
11 else:
12     print("Divisao por zero evitada com seguranca!")
```

Manipulação e Fatiamento de Strings (Slicing)

Strings são sequências imutáveis de caracteres.

```
1 texto = "Algoritmos Avancados"
2
3 # Fatiamento: string[inicio:fim_exclusivo:passo]
4 print(texto[0:10])          # Output: Algoritmos
5 print(texto[:10])           # Output: Algoritmos (inicio padrao = 0)
6 print(texto[11:])           # Output: Avancados
7 print(texto[::-1])          # Inverte a string!
8
9 # Metodos essenciais
10 dados = "  ana;20;8.5  "
11 limpo = dados.strip().split(";") # ['ana', '20', '8.5']
12 formatado = ", ".join(limpo)     # 'ana, 20, 8.5'
```

Entrada e Saída Formatada com f-strings

```
1 nome = "AgroData"
2 produtividade = 12450.789
3 ativo = True
4
5 # Formatacao de numeros e alinhamento
6 print(f"Empresa: {nome:>15}")           # Alinhado a direita (width
    =15)
7 print(f"Produtividade: {produtividade:,.2f} kg") # 2 casas decimais e
    separador
8 print(f"Status: {'Ativo' if ativo else 'Inativo'}")
9
10 # Depuracao rapida com f-strings (Python 3.8+)
11 x = 10
12 y = 25
13 print(f"{x=}, {y=}, {x+y=}") # Output: x=10, y=25, x+y=35
```

Estruturas Condicionais (if, elif, else)

```
1 media = float(input("Informe a media final: "))
2 frequencia = float(input("Informe a frequencia (0 a 100%): "))
3
4 if frequencia < 75.0:
5     print("Reprovado por Frequencia (RF)")
6 elif media >= 6.0:
7     print("Aprovado por Nota e Frequencia (AP)")
8 elif media >= 4.0:
9     print("Em Exame / Prova Substitutiva (EX)")
10 else:
11     print("Reprovado por Nota (RN)")
```

Expressões Ternárias (Atribuição Condicional)

Permitem atribuir valores de forma enxuta baseando-se em uma condição.

```
1 # Sintaxe: valor_se_verdadeiro if condicao else valor_se_falso
2 idade = 19
3 status = "Maior de idade" if idade >= 18 else "Menor de idade"
4
5 # Exemplo em calculo de algoritmo
6 saldo = 1500.0
7 taxa = 0.02 if saldo > 1000.0 else 0.05
8 print(f"Taxa aplicada: {taxa * 100}%")
```

Pattern Matching (match / case - Python 3.10+)

Estrutura moderna equivalente ao switch-case de outras linguagens.

```
1 comando = "iniciar"
2
3 match comando:
4     case "iniciar" | "start":
5         print("Iniciando processamento do algoritmo...")
6     case "parar" | "stop":
7         print("Encerrando execucao.")
8     case "status":
9         print("Servico ativo.")
10    case _:
11        print("Comando nao reconhecido.")
```

Estrutura de Repetição while

O laço while executa enquanto uma condição booleana for verdadeira.

```
1 # Leitura e validacao continua de dados
2 nota = -1.0
3
4 while nota < 0.0 or nota > 10.0:
5     nota = float(input("Digite uma nota valida (0 a 10): "))
6     if nota < 0.0 or nota > 10.0:
7         print("Valor invalido! Tente novamente.")
8
9 print(f"Nota inserida com sucesso: {nota:.1f}")
```

Estrutura de Repetição for

O laço for no Python é focado na **iteração sobre sequências e iteráveis**.

```
1 # Iterando sobre uma lista de dados do agronegocio
2 culturas = ["Soja", "Milho", "Algodao", "Cafe"]
3
4 for cultura in culturas:
5     print(f"Cultura cadastrada: {cultura}")
6
7 # Iterando sobre os caracteres de uma string
8 sigla = "FATEC"
9 for letra in sigla:
10     print(letra, end="-")    # F-A-T-E-C-
```

A Função range()

A função range() gera uma sequência numérica de forma eficiente na memória.

```
1 # range(inicio, fim_exclusivo, passo)
2
3 # 1. Somente fim (inicio=0, passo=1)
4 for i in range(3):
5     print(i, end=" ") # Output: 0 1 2
6
7 # 2. Inicio e Fim
8 for i in range(5, 8):
9     print(i, end=" ") # Output: 5 6 7
10
11 # 3. Inicio, Fim e Passo (Contagem Regressiva)
12 for i in range(10, 0, -2):
13     print(i, end=" ") # Output: 10 8 6 4 2
```

Cláusula else em Laços (for-else e while-else)

O bloco else de um laço executa ****apenas se o laço terminar normalmente**** (sem encontrar um break).

```
1 numeros = [2, 4, 6, 8, 10]
2 alvo = 7
3
4 for num in numeros:
5     if num == alvo:
6         print(f"Elemento {alvo} encontrado!")
7         break
8 else:
9     # Executado apenas se a busca percorrer toda a lista sem encontrar o
    alvo
10    print(f"Elemento {alvo} nao esta presente na lista.")
```

Controle de Fluxo Interno: break, continue e pass

break

Interrompe e sai do laço imediatamente.

continue

Pula a iteração atual e vai para a próxima.

pass

Instrução nula (*placeholder* de sintaxe).

```
1 for i in range(1, 6):  
2     if i == 3:  
3         continue    # Pula o 3  
4     if i == 5:  
5         break        # Para no 5  
6     print(i, end=" ") # Output: 1 2 4
```

Iteração Elegante I: enumerate()

Acessa o índice e o elemento da coleção simultaneamente sem precisar controlar contadores manuais.

```
1 sensores = ["Temperatura", "Umidade", "Pressao", "PH do Solo"]
2
3 # enumerate(colecao, start=0)
4 for indice, nome_sensor in enumerate(sensores, start=1):
5     print(f"Sensor ID #{indice:02d}: {nome_sensor}")
6
7 # Output:
8 # Sensor ID #01: Temperatura
9 # Sensor ID #02: Umidade ...
```

Iteração Elegante II: zip()

Permite iterar sobre duas ou mais sequências em paralelo.

```
1 talhoes = ["T-01", "T-02", "T-03"]
2 areas_ha = [120.5, 85.0, 200.3]
3 produtividades = [72.4, 68.0, 81.2]
4
5 for talhao, area, prod in zip(talhoes, areas_ha, produtividades):
6     total_ton = (area * prod) / 10
7     print(f"Talhao {talhao} | Area: {area} ha | Prod. Total: {total_ton:.1f}
8         t")
```

Desempacotamento de Sequências (*Unpacking*)

```
1 # Atribuicao Simultanea e Troca de Valores (Swap)  
2 a, b = 10, 20  
3 a, b = b, a # Troca elegante sem variavel auxiliar!  
4  
5 # Desempacotamento com operador rest (*)  
6 valores = [10, 20, 30, 40, 50]  
7 primeiro, *meio, ultimo = valores  
8  
9 print(primeiro)    # 10  
10 print(meio)       # [20, 30, 40]  
11 print(ultimo)     # 50
```

Tratamento Básico de Exceções (try / except)

Garante resiliência ao código evitando que entradas inválidas interrompam o programa.

```
1 try:
2     valor = float(input("Digite a produtividade estimada: "))
3     resultado = 1000 / valor
4 except ValueError:
5     print("Erro: Digite um numero valido!")
6 except ZeroDivisionError:
7     print("Erro: A produtividade nao pode ser zero!")
8 else:
9     print(f"Calculo realizado com sucesso: {resultado:.2f}")
10 finally:
11     print("Fim do bloco de leitura.")
```

Exercício Prático 1: Enunciado

Problema 1: Processamento de Notas com Validação

Escreva um programa Python que receba continuamente as notas de alunos de uma disciplina até que o usuário digite -1. O programa deve:

- 1 Validar as entradas, ignorando valores fora do intervalo $[0, 10]$ e tratando erros de digitação.
- 2 Calcular e exibir a média geral das notas válidas inseridas.
- 3 Exibir a maior e a menor nota registradas.

Exercício Prático 1: Solução Comentada

```
1 soma_notas, qtd_validas = 0.0, 0
2 maior_nota, menor_nota = float('-inf'), float('inf')
3
4 while True:
5     try:
6         entrada = float(input("Digite uma nota (ou -1 para encerrar): "))
7         if entrada == -1:
8             break
9         if not (0.0 <= entrada <= 10.0):
10            print("Nota invalida! Deve estar entre 0 e 10.")
11            continue
12        soma_notas += entrada
13        qtd_validas += 1
14        maior_nota = max(maior_nota, entrada)
15        menor_nota = min(menor_nota, entrada)
16    except ValueError:
17        print("Entrada invalida! Digite apenas numeros.")
18
19 if qtd_validas > 0:
20     print(f"Media: {soma_notas / qtd_validas:.2f} | Maior: {maior_nota} |
```



Exercício Prático 2: Enunciado

Problema 2: Teste de Primalidade

Escreva uma função em Python `eh_primo(n: int) -> bool` que determine se um número inteiro positivo n é primo ou não.

- Compare a eficiência de testar todos os números até $n - 1$ versus testar apenas até $\sqrt{n}$.

Exercício Prático 2: Abordagem vs. Otimização

Força Bruta $O(n)$

```
1 def eh_primo_lento(n: int) -> bool:
2     if n <= 1:
3         return False
4     for i in range(2, n):
5         if n % i == 0:
6             return False
7     return True
```

Otimizado $O(\sqrt{n})$

```
1 def eh_primo_rapido(n: int) -> bool:
2     if n <= 1:
3         return False
4     # Testa apenas ate a raiz
5     quadrada de n
6     limite = int(n**0.5) + 1
7     for i in range(2, limite):
8         if n % i == 0:
9             return False
10    return True
```

Por que testar até $\sqrt{n}$ funciona?

Fundamentação Matemática

Se $n = a \times b$, então pelo menos um dos fatores (a ou b) deve ser menor ou igual a $\sqrt{n}$.

- Se não encontramos nenhum divisor até $\sqrt{n}$, é impossível existir um divisor maior que $\sqrt{n}$.
- Para $n = 1.000.000$:
 - Força Bruta: faz até **1.000.000 de operações**.
 - Algoritmo Otimizado: faz no máximo **1.000 operações**!

Exercício Prático 3: Enunciado

Problema 3: Contagem e Análise de Frequências

Escreva um programa Python que receba uma lista de leituras numéricas de produtividade por hectare de diferentes talhões e identifique:

- 1 A produtividade média geral.
- 2 Quantos talhões ficaram acima da média estimada.

Exercício Prático 3: Solução Comentada

```
1 leituras = [65.4, 72.0, 58.5, 80.2, 74.1, 69.8, 85.0, 61.2]
2
3 # 1. Calculo da media geral
4 media = sum(leituras) / len(leituras)
5
6 # 2. Contagem de talhoes acima da media usando gerador/condicional
7 acima_da_media = [p for p in leituras if p > media]
8 qtd_acima = len(acima_da_media)
9
10 print(f"Produtividade Media: {media:.2f} sacas/ha")
11 print(f"Talhoes acima da media: {qtd_acima} de {len(leituras)}")
12 print(f"Valores acima: {acima_da_media}")
```

Próximos Passos

Conteúdo da Próxima Aula (Aula 03)

- **Tipos Abstratos de Dados (TADs) – Listas em Python**
- Criação, indexação, fatiamento (*slicing*) e métodos nativos de listas.
- List Comprehensions e Eficiência na Manipulação de Sequências.

Tarefa Complementar

Revisar o ambiente Python local (VS Code / PyCharm / Jupyter Notebook) e resolver a Lista 1 de exercícios no repositório oficial da disciplina.

Bons Estudos!

Dúvidas ou Perguntas?